

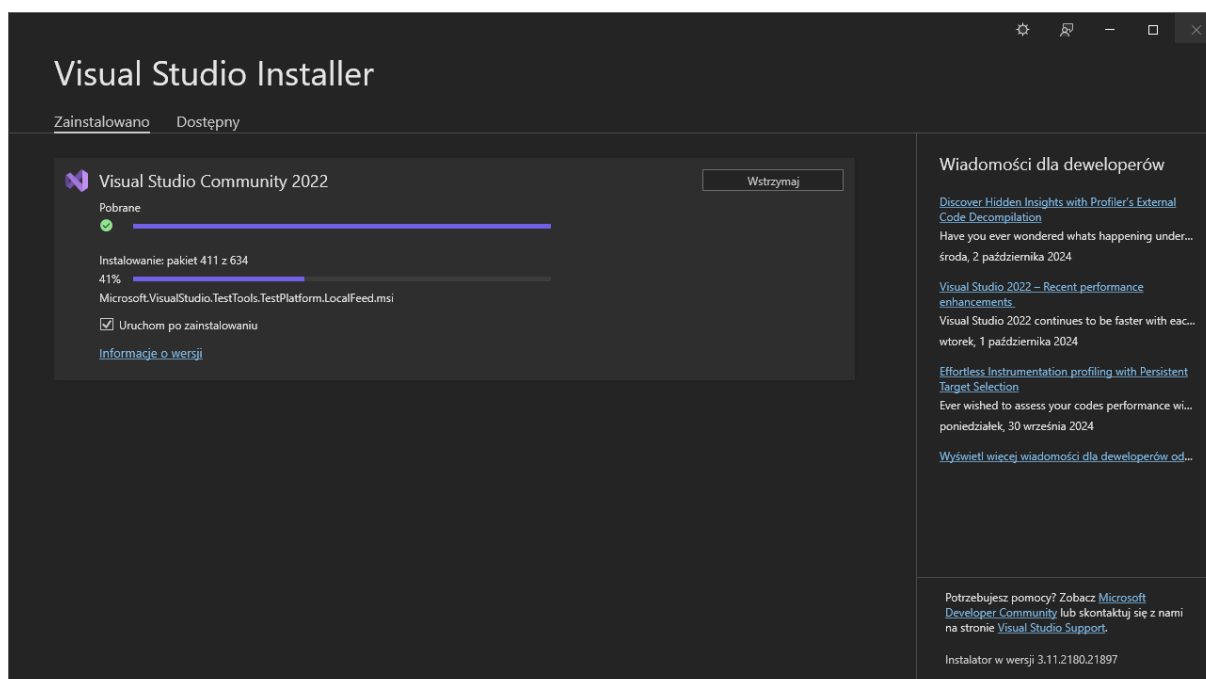
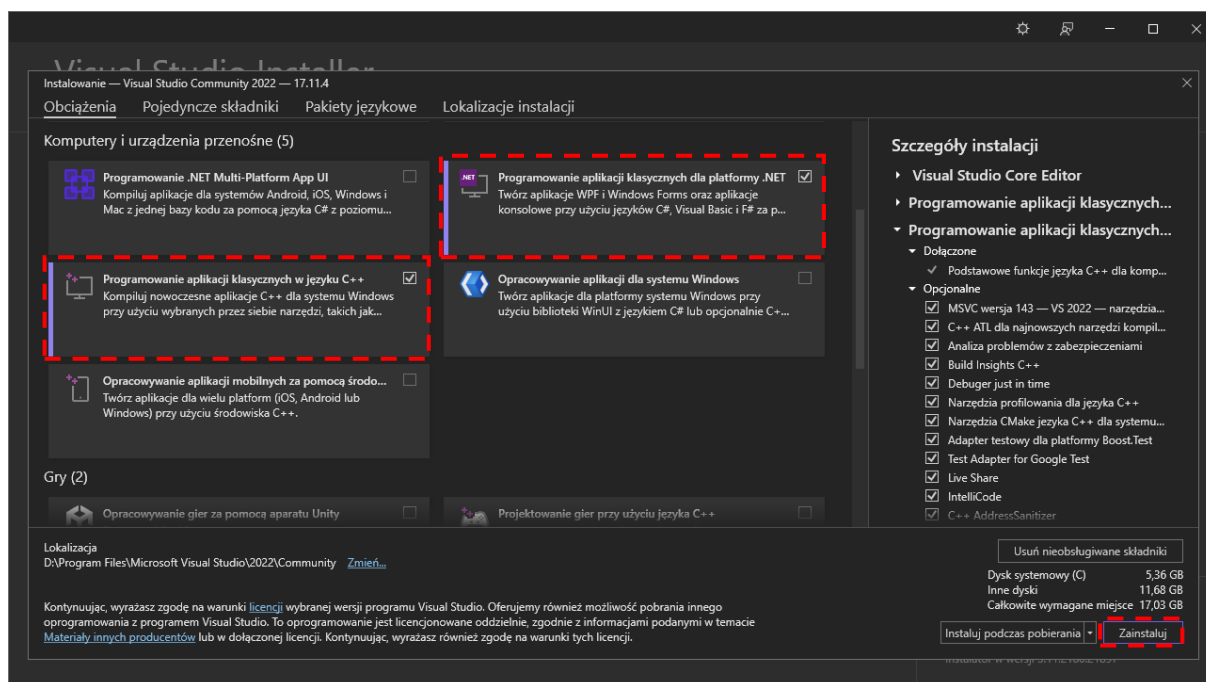
Jeśli na stanowisku nie ma zainstalowanego programu Visual Studio to pracę rozpoczynamy od procesu instalacji, który znajduje się poniżej (krok 1). Jeśli program jest zainstalowany należy przejść do następnego kroku (nr 2).

1. Instalacja Visual Studio.

Instalator programu Visual Studio jest dostępny pod adresem:

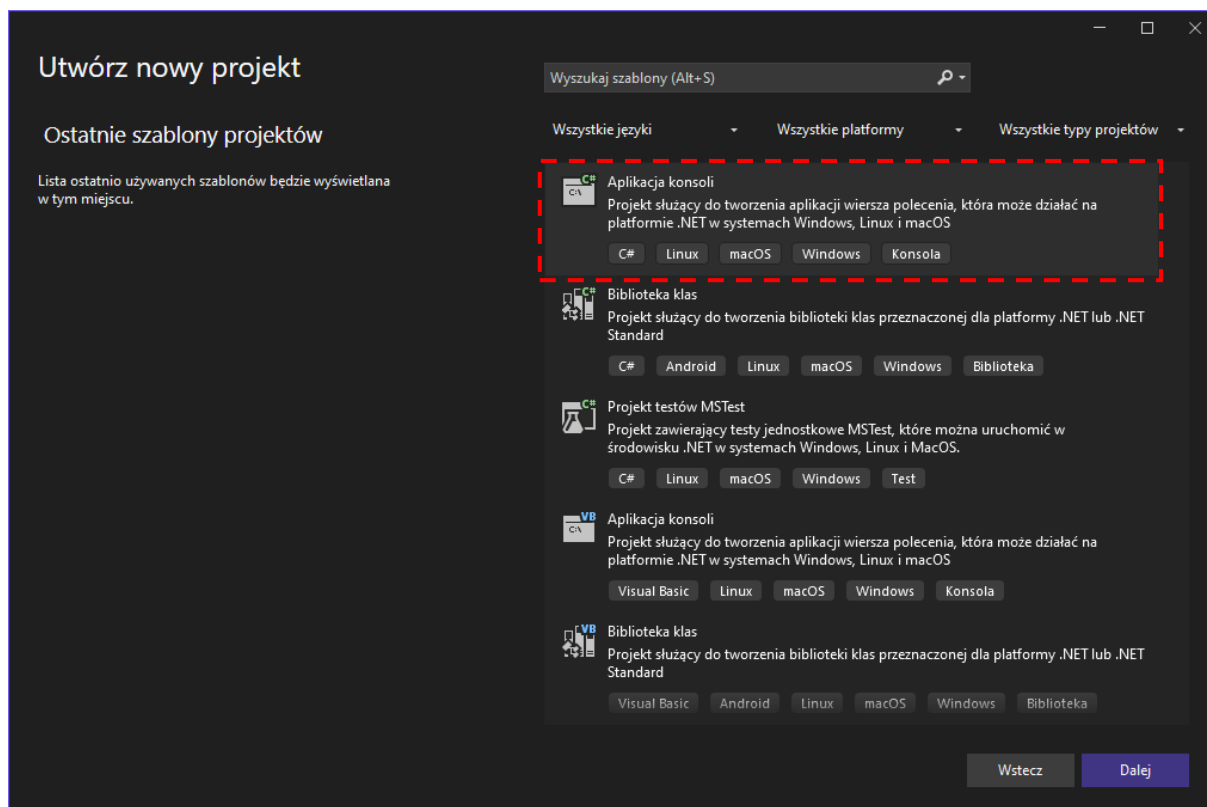
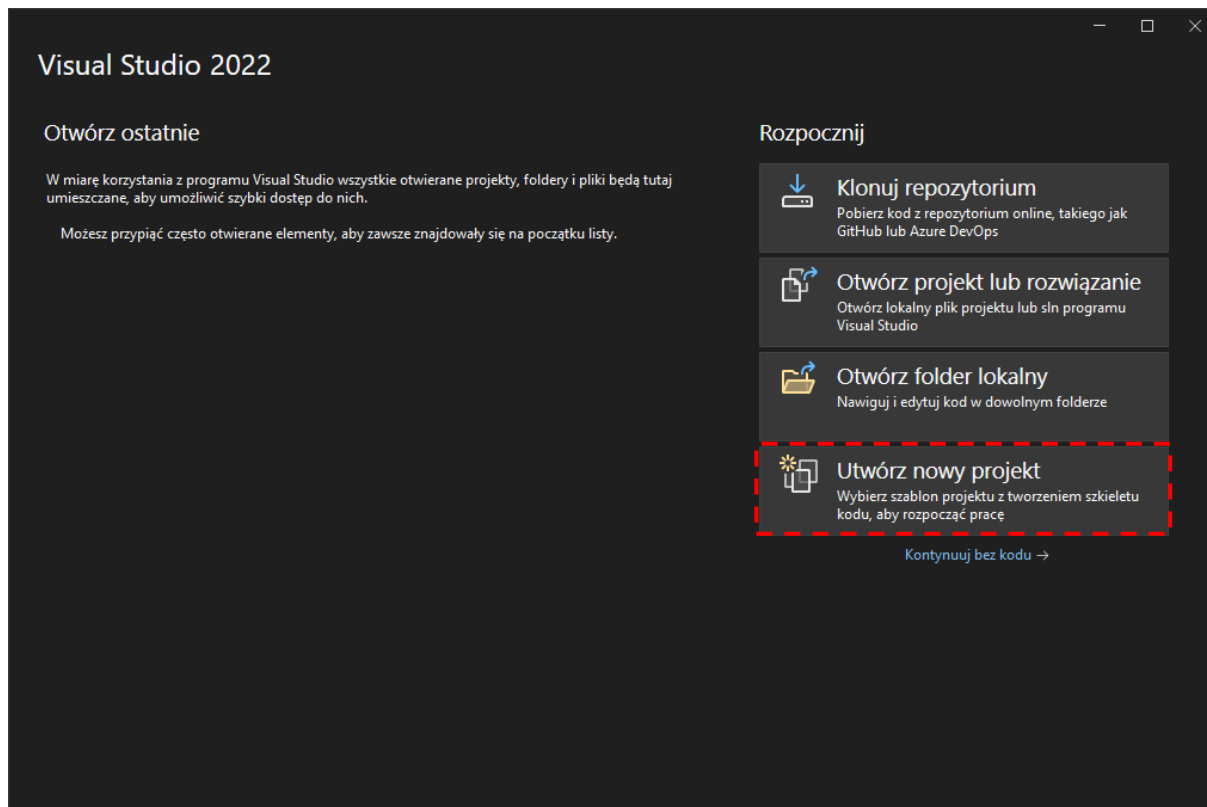
<https://visualstudio.microsoft.com/pl/free-developer-offers/>

Po uruchomieniu instalatora wybieramy potrzebne pakiety i klikamy zainstaluj:

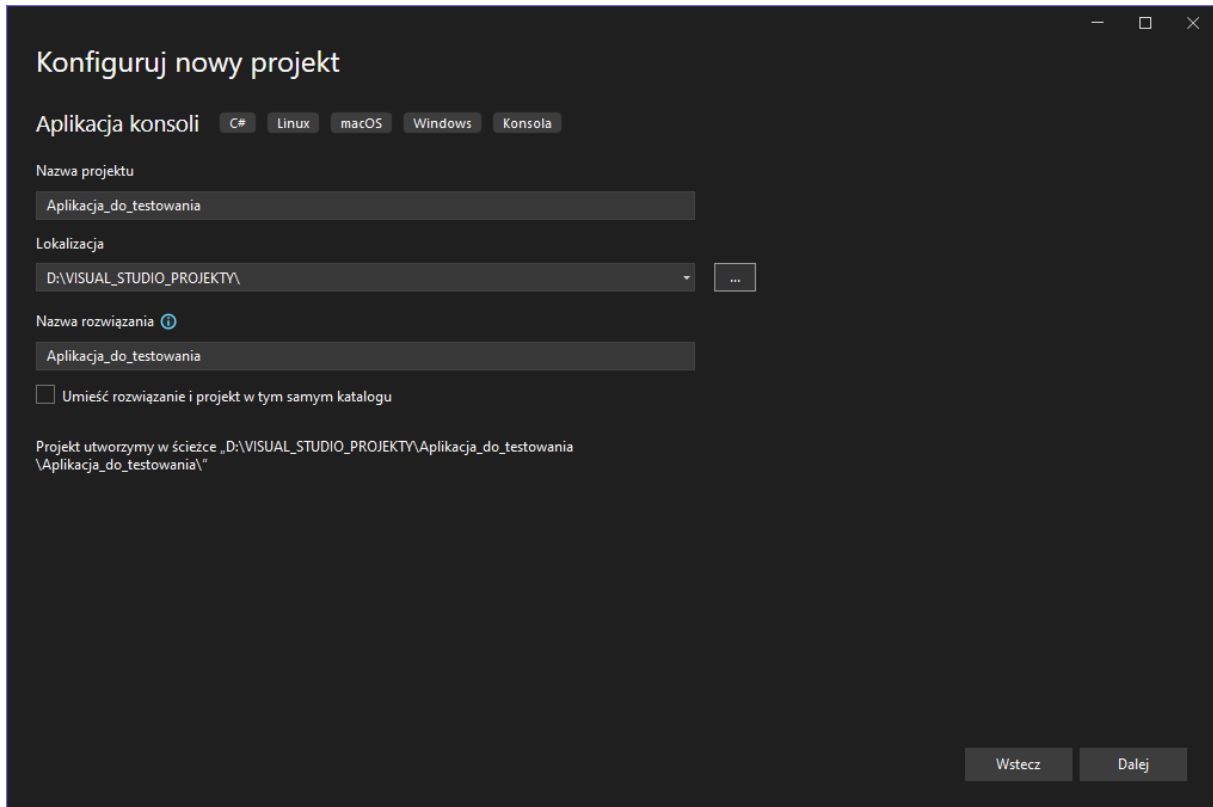


2. Utworzenie projektu C#.

Po uruchomieniu programu Visual Studio wybieramy opcję „Utwórz nowy projekt”, a następnie zaznaczamy szablon „Aplikacja konsoli C#” i klikamy przycisk „dalej”:



Wpisujemy nazwę naszego projektu:



Konfiguruj nowy projekt

Aplikacja konsoli C# Linux macOS Windows Konsola

Nazwa projektu

Aplikacja_do_testowania

Lokalizacja

D:\VISUAL_STUDIO_PROJEKTY\...

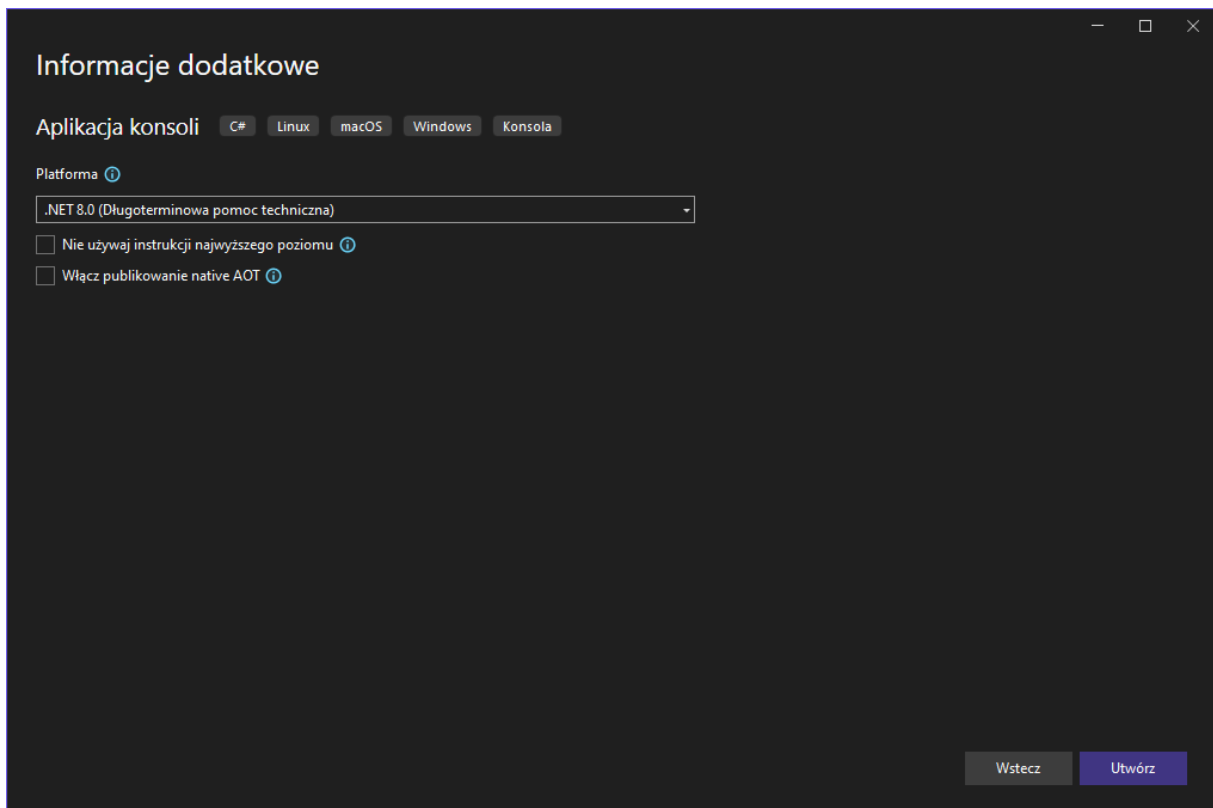
Nazwa rozwiązania ⓘ

Aplikacja_do_testowania

☐ Umieść rozwiązanie i projekt w tym samym katalogu

Projekt utworzymy w ścieżce „D:\VISUAL_STUDIO_PROJEKTY\Aplikacja_do_testowania\Aplikacja_do_testowania\”

Wstecz Dalej



Informacje dodatkowe

Aplikacja konsoli C# Linux macOS Windows Konsola

Platforma ⓘ

.NET 8.0 (Długoterminowa pomoc techniczna)

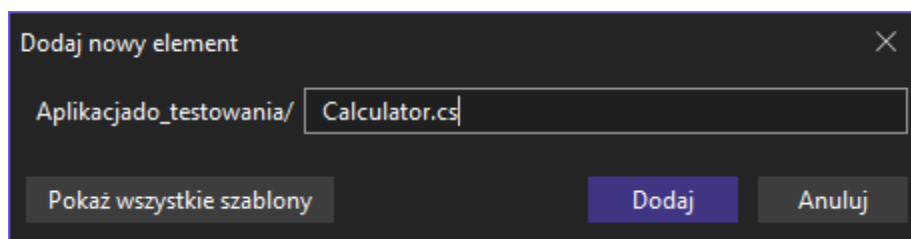
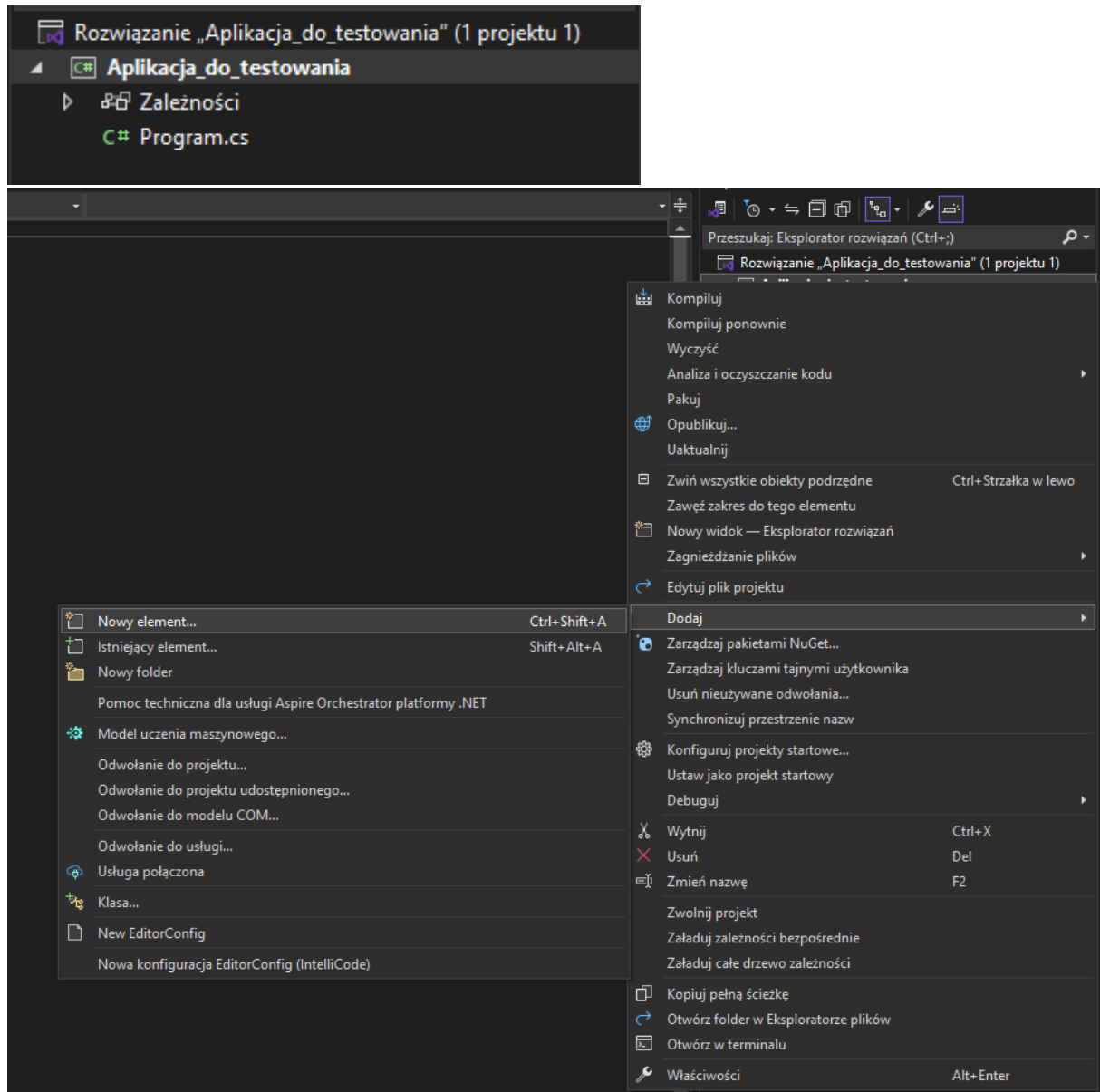
☐ Nie używaj instrukcji najwyższego poziomu ⓘ

☐ Włącz publikowanie native AOT ⓘ

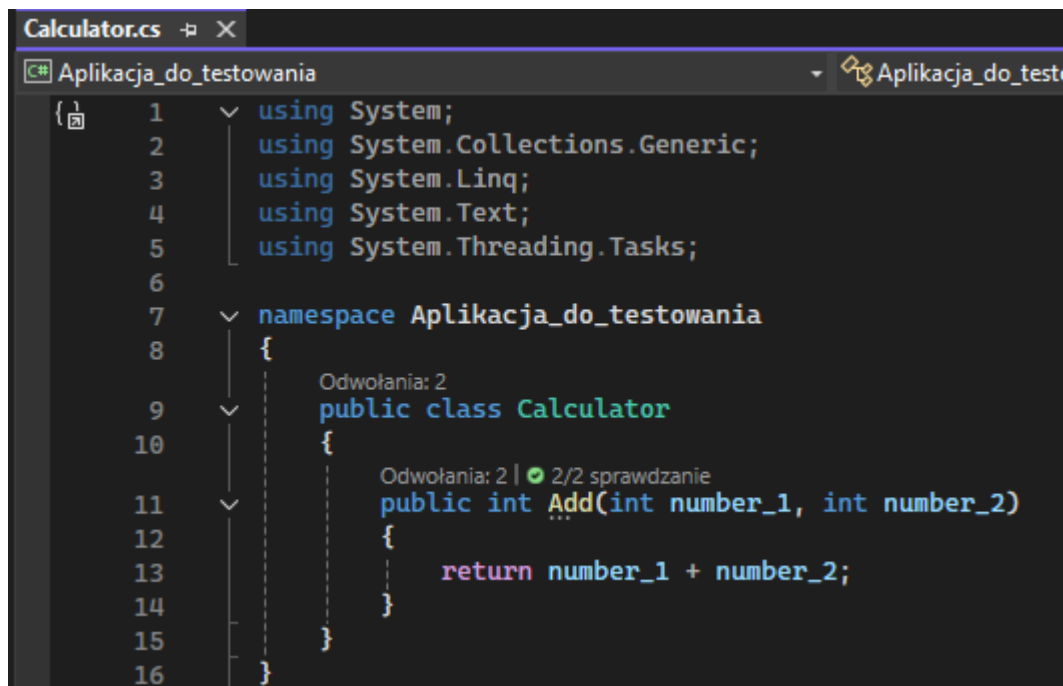
Wstecz Utwórz

3. Napisanie prostego programu C#.

W drzewku po prawej stronie (Eksplorator rozwiązań) klikamy prawym przyciskiem myszy na nazwę projektu. Następnie wybieramy Dodaj -> Nowy element.



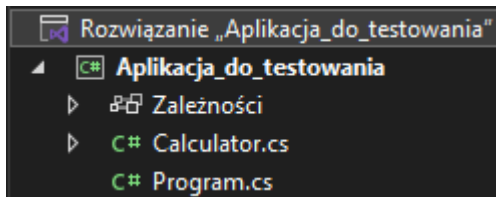
W utworzonym pliku (w tym przypadku Calculator.cs) implementujemy prostą funkcję dodawania:



```
Calculator.cs  [icon] X
Aplikacja_do_testowania
1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using System.Text;
5  using System.Threading.Tasks;
6
7  namespace Aplikacja_do_testowania
8  {
9      public class Calculator
10     {
11         public int Add(int number_1, int number_2)
12         {
13             return number_1 + number_2;
14         }
15     }
16 }
```

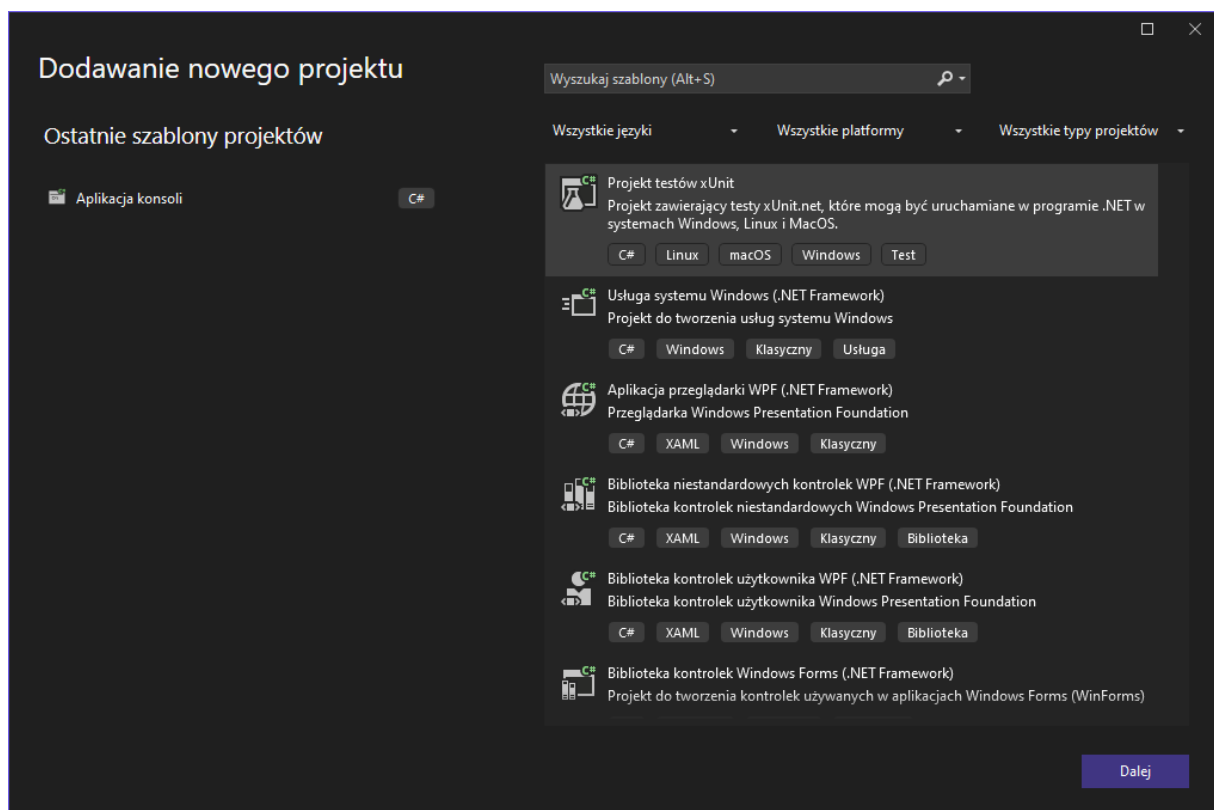
4. Dodanie i uruchomienie testów xUnit.

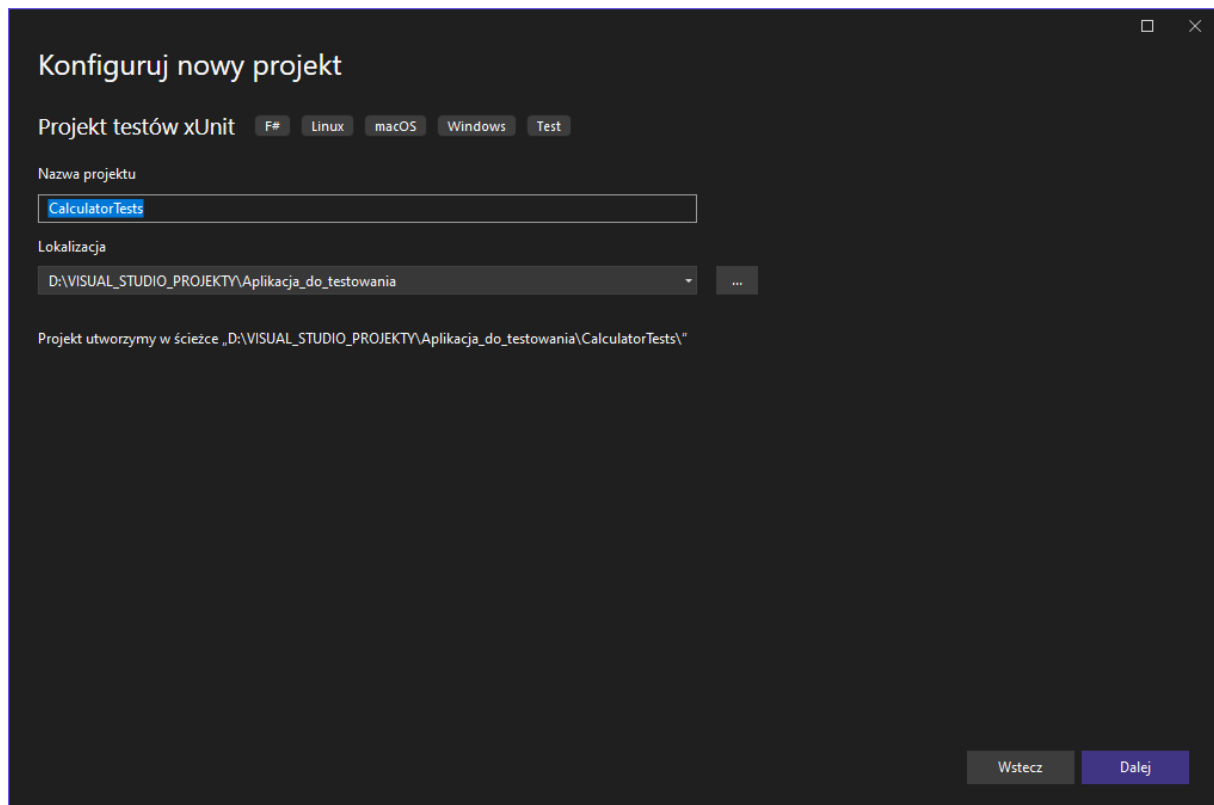
Klikamy prawym przyciskiem myszy na Rozwiązanie „Nazwa projektu”:



Następnie wybieramy Dodaj -> Nowy projekt

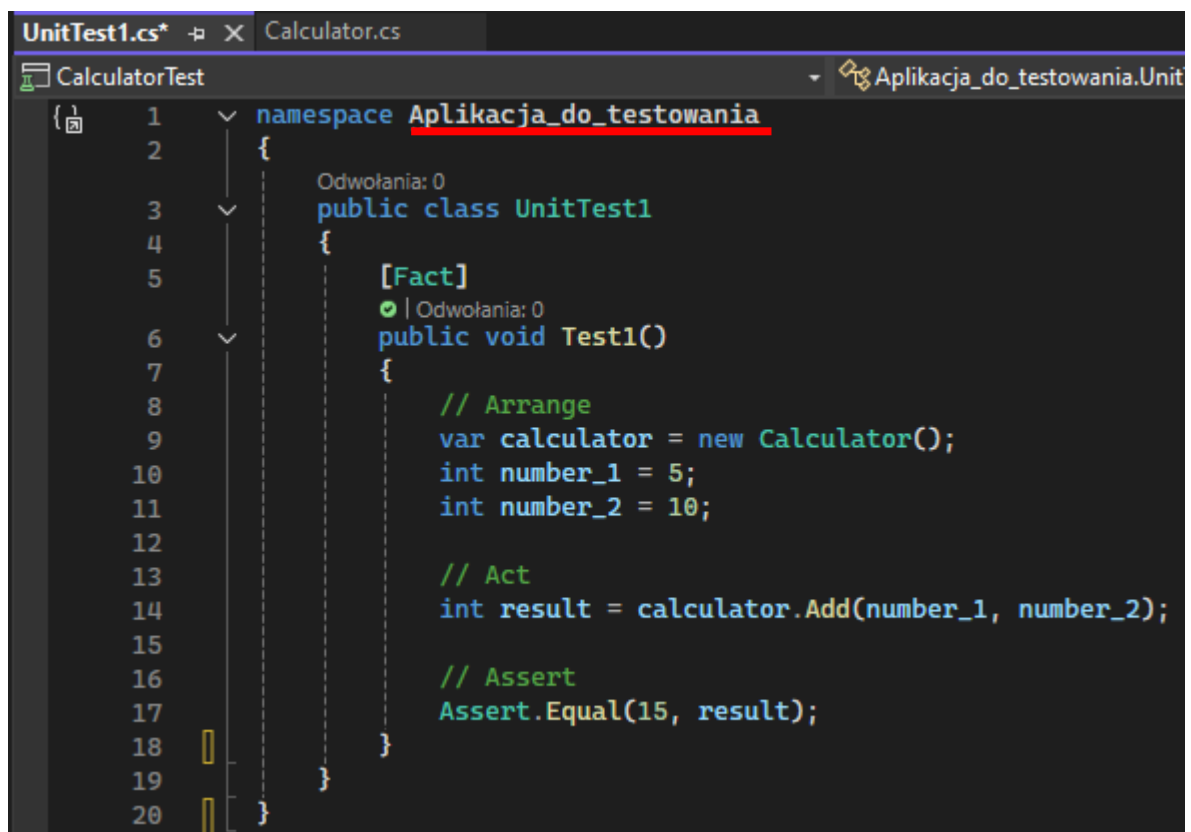
W okienku dodawania projektu wybieramy pozycję „Projekt testów xUnit C#”, a w kolejnym etapie wpisujemy jego nazwę.



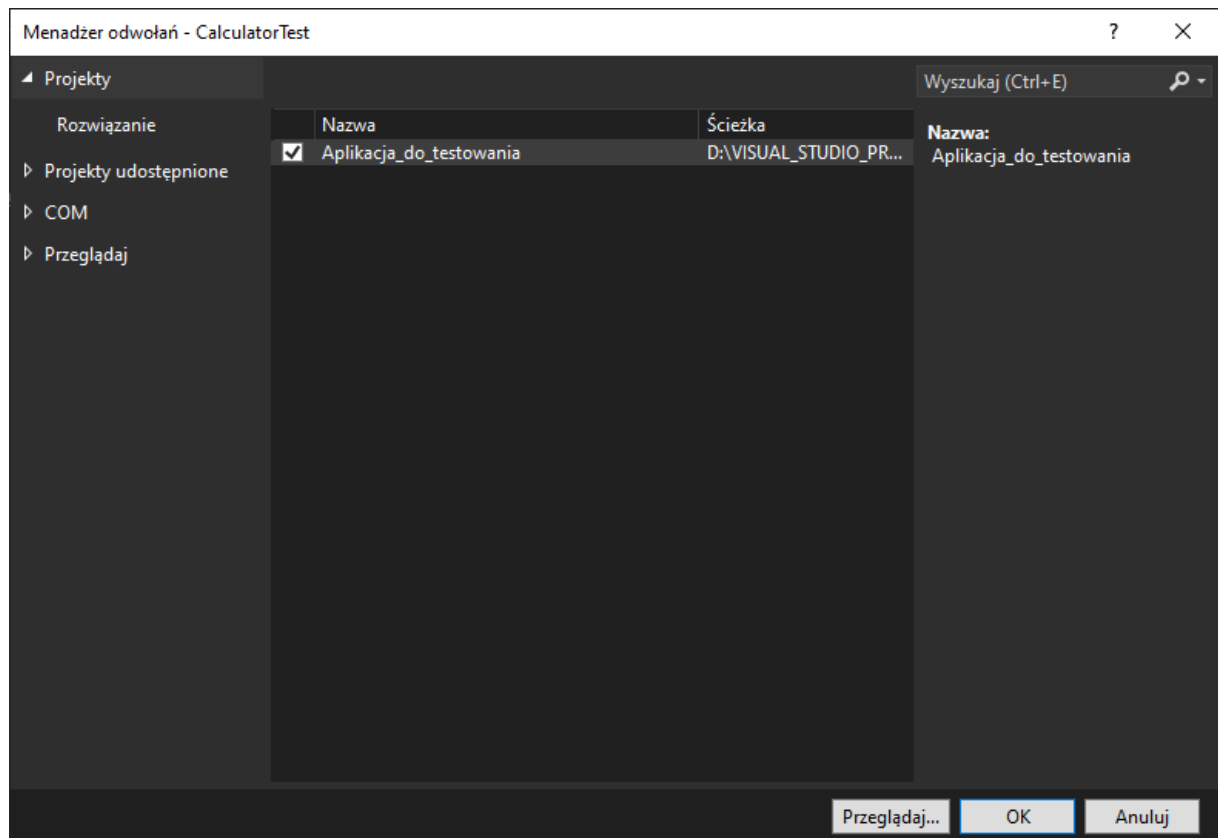


W nowo utworzonym projekcie (w tym przypadku CalculatorTest) tworzymy przykładowy test o nazwie Test1():

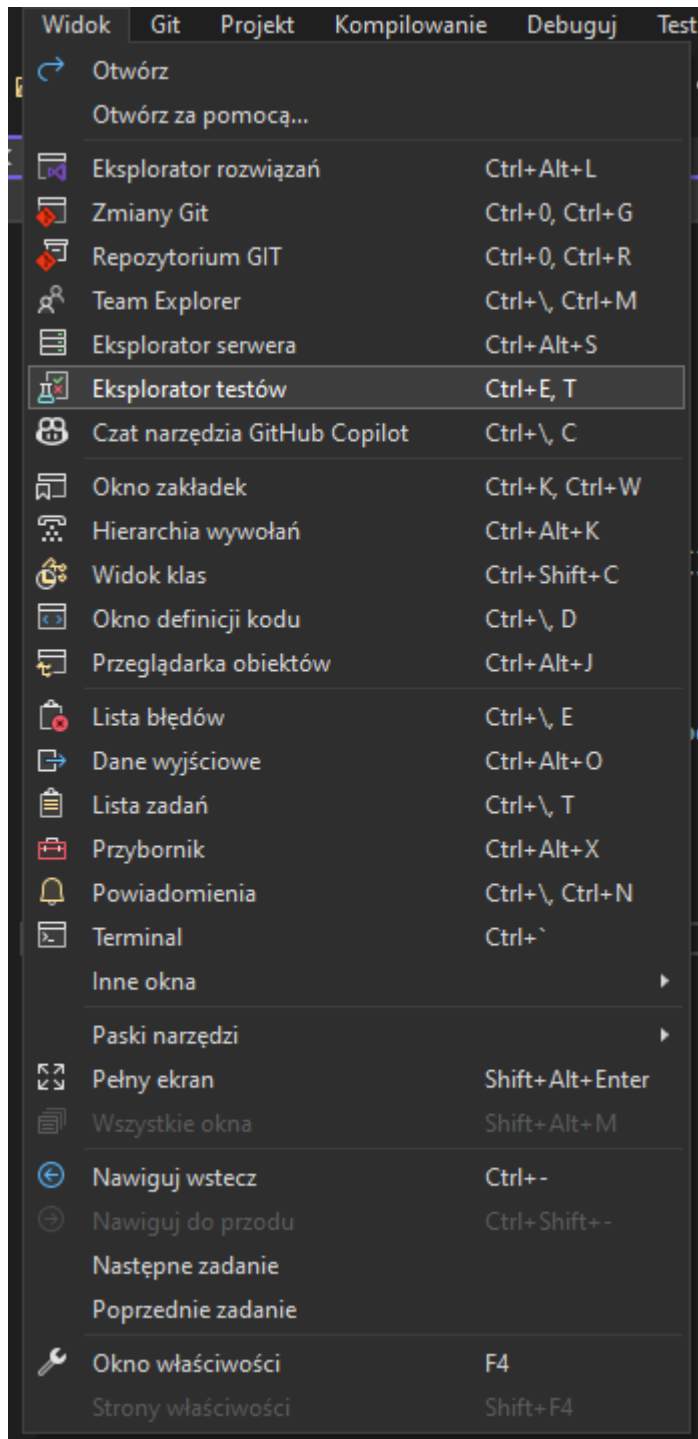
Pamiętamy o użyciu tej samej przestrzeni nazw co w pliku Calculator.cs !!!



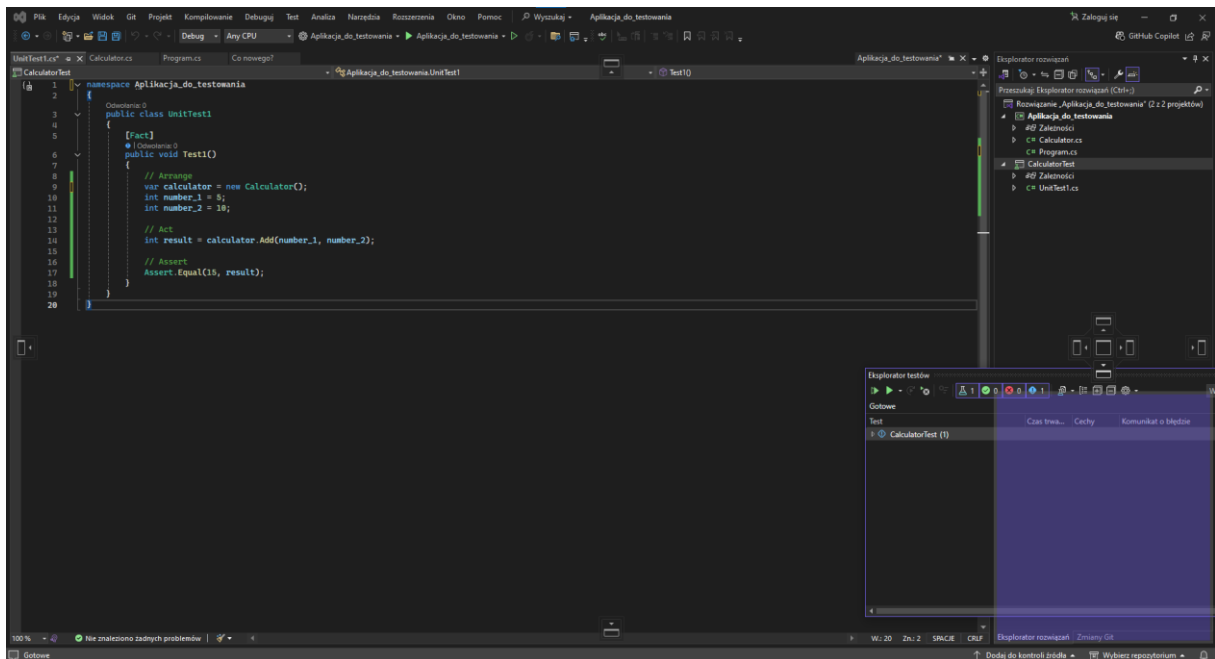
Kolejnym etapem jest dodanie odwołania do projektu:



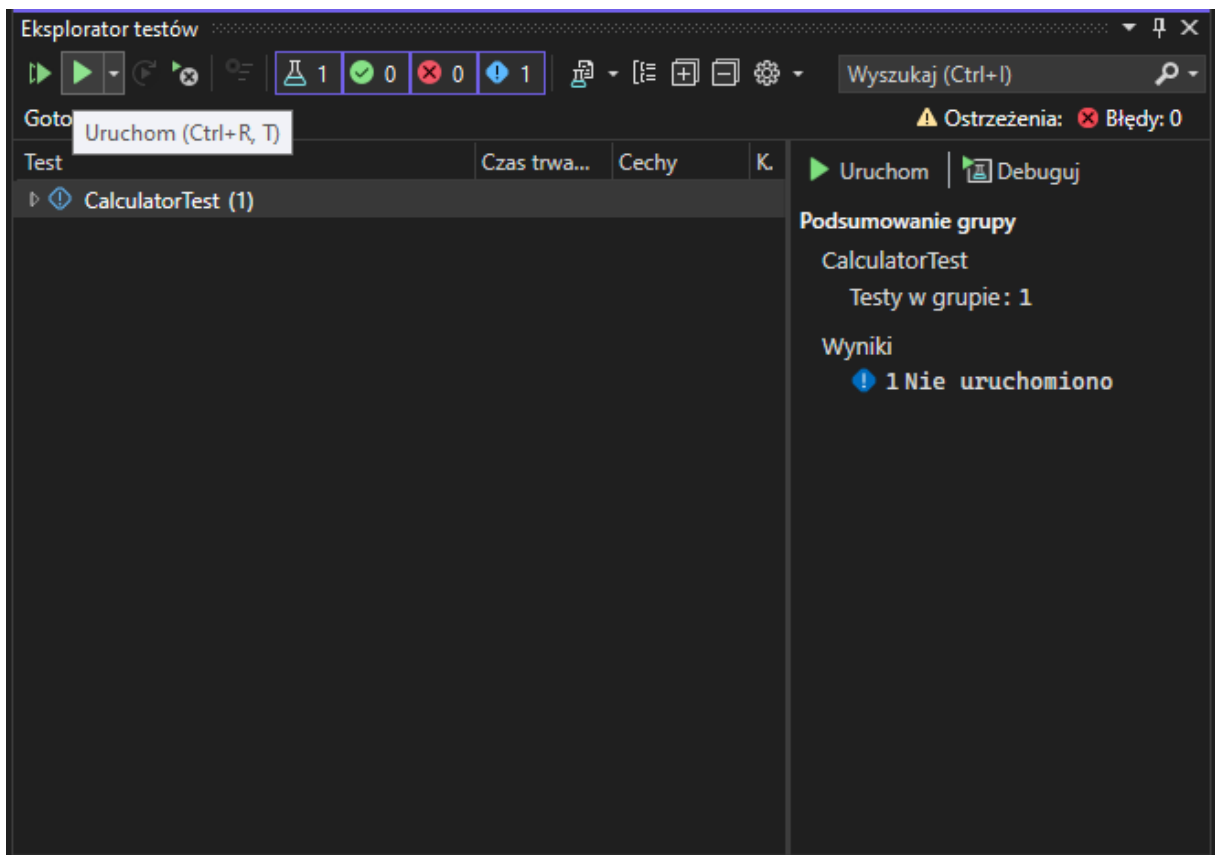
W celu uruchomienia testów wybieramy zakładkę widok, a następnie „Eksplorator testów”.

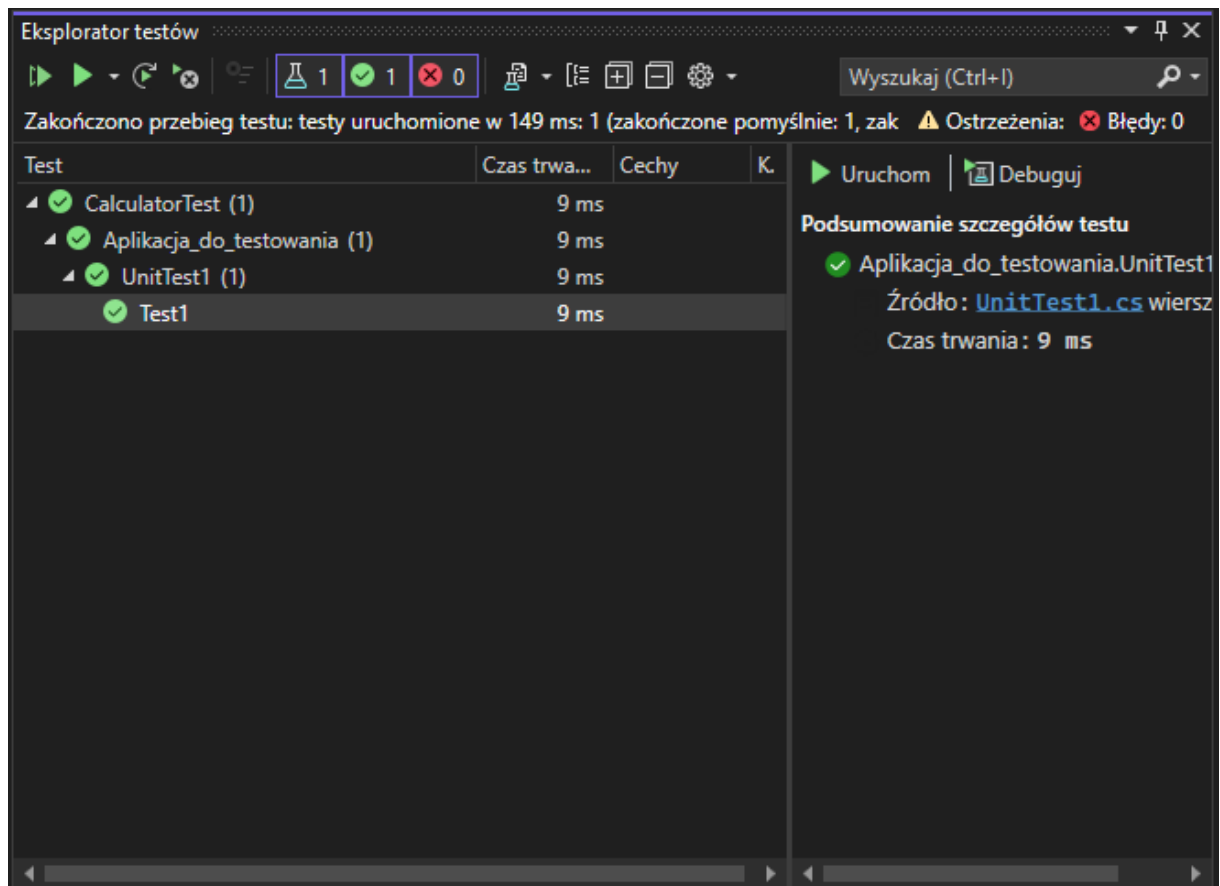


Dla wygody okienko z testami można umieścić we wskazanym miejscu.

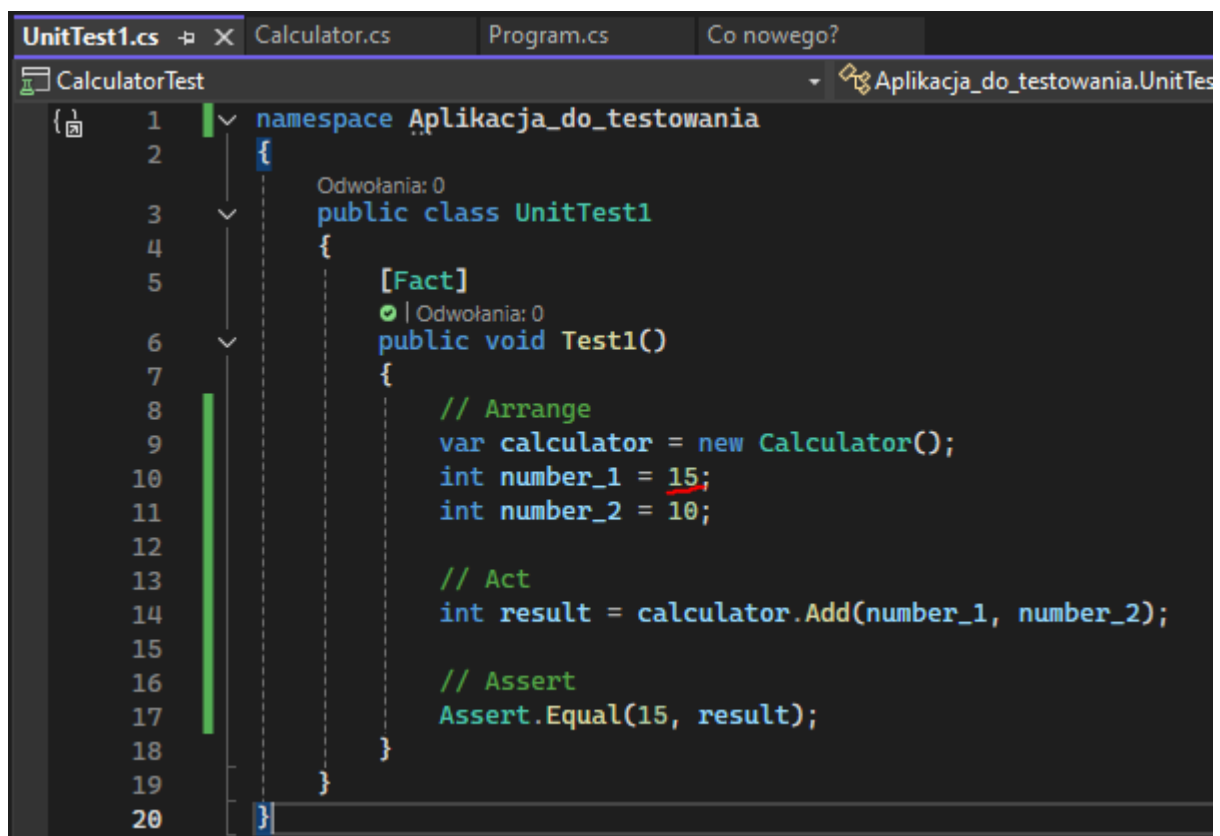


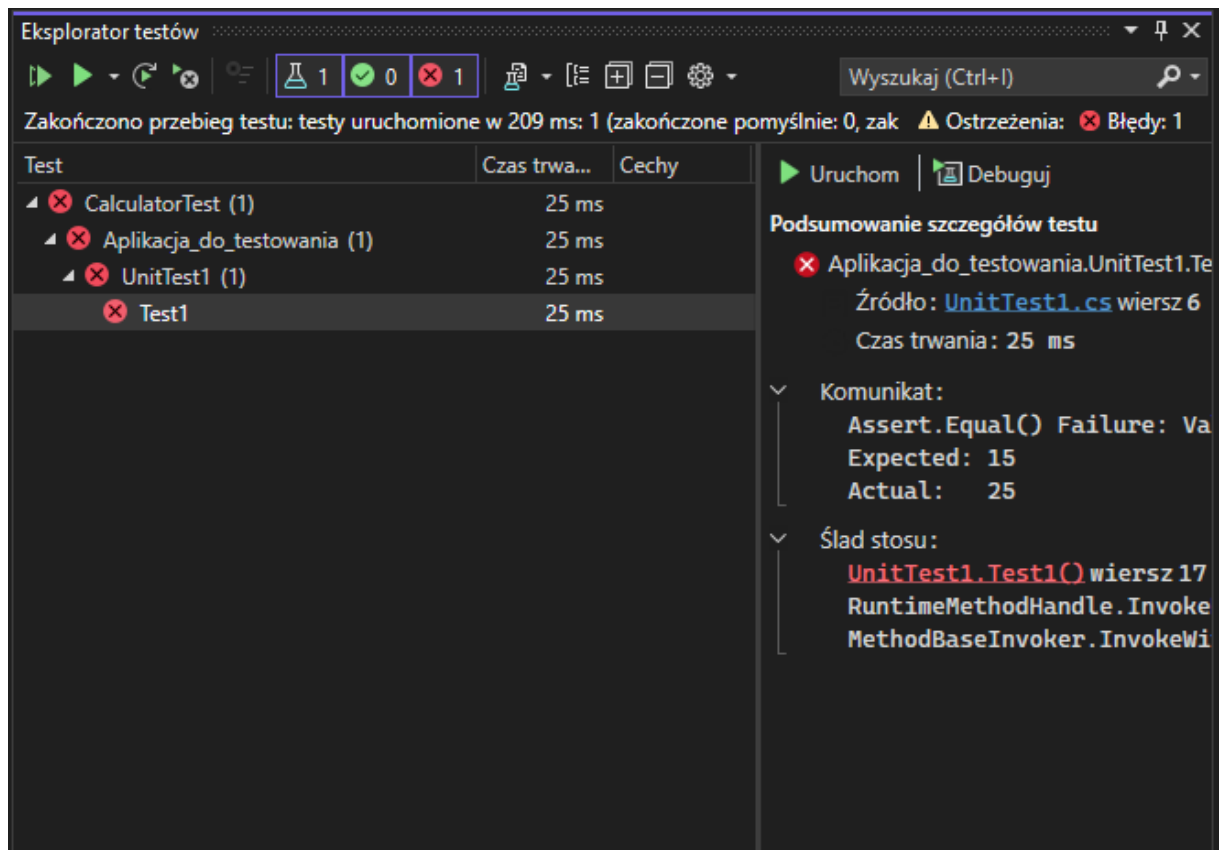
Aby wykonać testy wybieramy opcję uruchom:





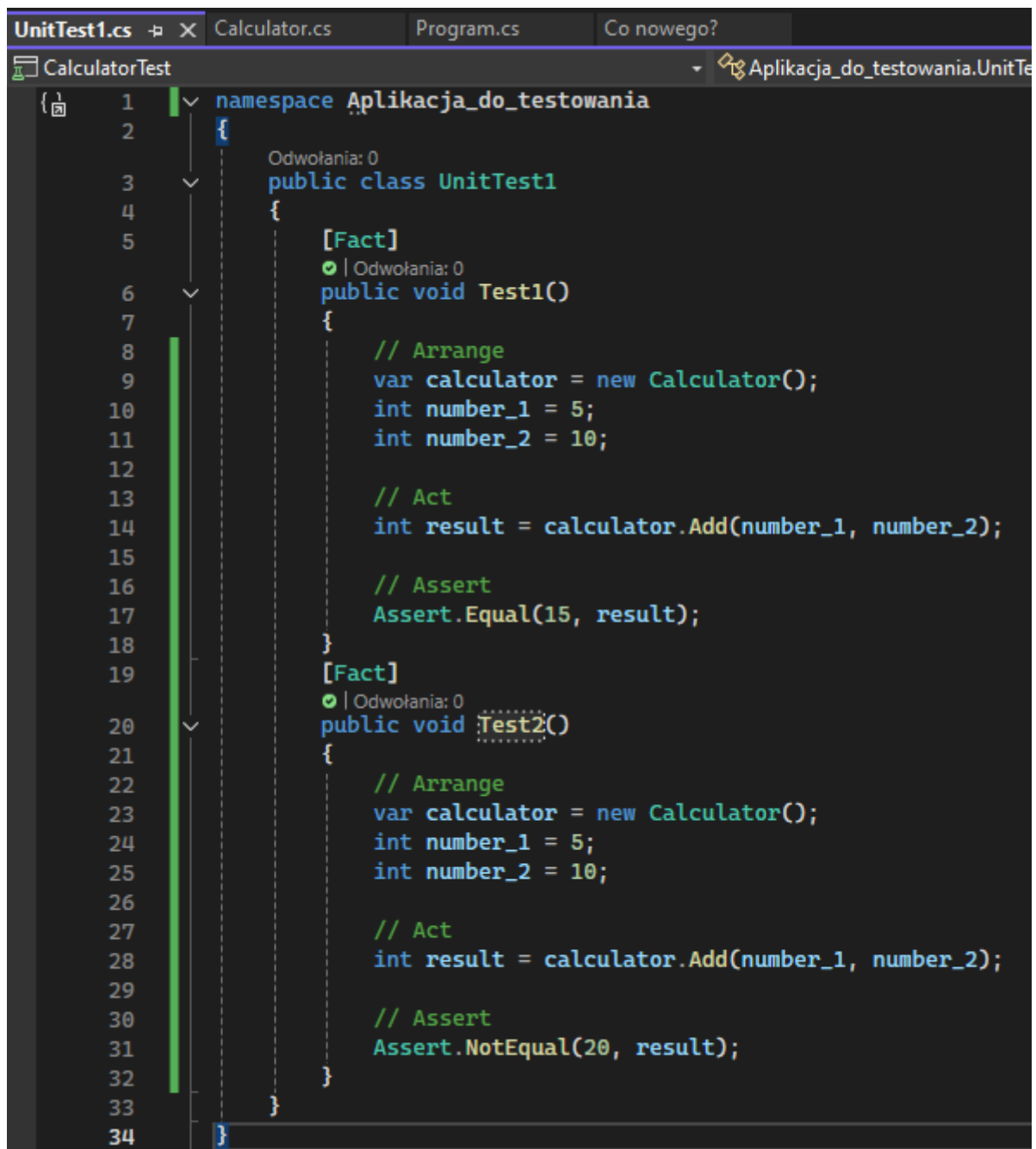
Po wprowadzeniu celowego błędu w kodzie uruchamiamy test ponownie.



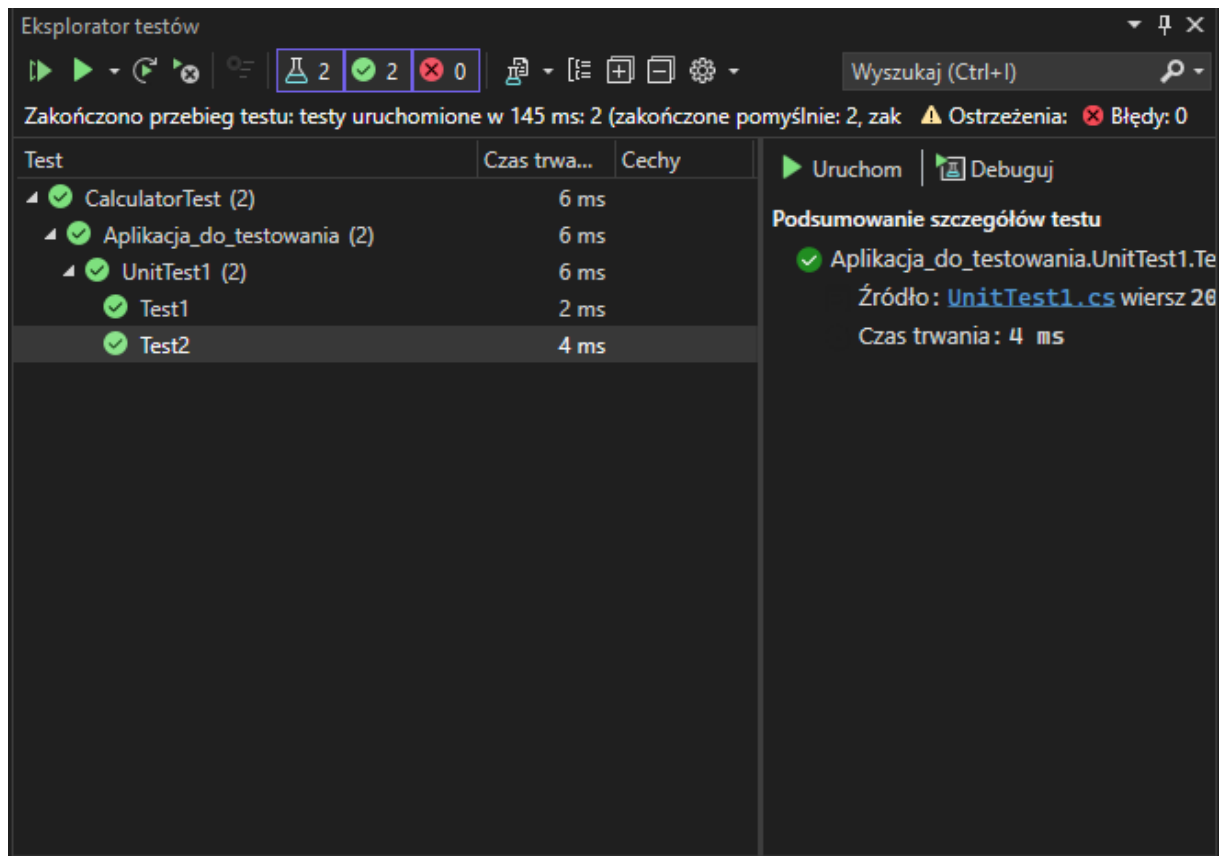


Test zakończył się niepowodzeniem. W widocznym po prawej stronie komunikacie widzimy wartość jakiej test oczekiwał (15) oraz wartość jaką otrzymał (25).

Stwórzmy jeszcze jeden test. Test2 będzie testem negatywnym (co nie oznacza że ma się zakończyć niepowodzeniem) w którym wykorzystamy `Assert.NotEqual` sprawdzając zachowanie programu przy celowo ustawionej złej wartości (wartość 20).



```
1 namespace Aplikacja_do_testowania
2 {
3     Odwołania: 0
4     public class UnitTest1
5     {
6         [Fact]
7         Odwołania: 0
8         public void Test1()
9         {
10             // Arrange
11             var calculator = new Calculator();
12             int number_1 = 5;
13             int number_2 = 10;
14
15             // Act
16             int result = calculator.Add(number_1, number_2);
17
18             // Assert
19             Assert.Equal(15, result);
20         }
21
22         [Fact]
23         Odwołania: 0
24         public void Test2()
25         {
26             // Arrange
27             var calculator = new Calculator();
28             int number_1 = 5;
29             int number_2 = 10;
30
31             // Act
32             int result = calculator.Add(number_1, number_2);
33
34             // Assert
35             Assert.NotEqual(20, result);
36         }
37     }
38 }
```



Wszystkie wykonywane testy muszą zakończyć się powodzeniem!

Na koniec warto wspomnieć że nazywanie testów Test1 i Test2 nie jest dobrą praktyką. Nazwa testu powinna skrótowo opisywać czego on dotyczy. Poprawione nazwy testów wyglądają następująco:

Test	Czas trwania	Cechy
CalculatorTest (2)	5 ms	
Aplikacja_do_testowania (2)	5 ms	
UnitTest1 (2)	5 ms	
Add_TwoPositiveNumbers_NotEqualToIncorrectSum	< 1 ms	
Add_TwoPositiveNumbers_ReturnsCorrectSum	5 ms	

ZADANIE:

Napisz nietrywialny program (przykład podam na zajęciach) w C# i utwórz do niego testy (przy użyciu xUnit) minimum 3 pozytywne i 1 negatywny. Podczas wykonywania programu i testów pamiętaj o robieniu commitów.